

Projeto 2 – Guia passo a passo

Estufa inteligente na plataforma FIWARE

Departamento de Ciência da Computação

Centro Universitário FEI

JESUÍTAS BRASIL

Como usar este guia

1

Plataforma

Define a API key, sobe o FIWARE e testa a ponte

`plataforma/`

2

Provisionamento

Corpos do grupo de serviço e do dispositivo

`postman/` · HiveMQ Web Client

3

Firmware

Migra o firmware do Projeto 1 para o IoT Agent

`firmware/main_esqueleto.py`

4

Consumo

Corpos das subscrições e o painel no Grafana

`postman/` · Grafana

Cada etapa termina com um **ponto de verificação**: a pasta da etapa na coleção Postman deve ficar **verde**. Entre as Etapas 1 e 2, rode o **exemplo resolvido** da lâmpada.

O guia diz **o que** fazer e **como conferir**. O **como fazer** das partes centrais é do grupo, com base nas Aulas 5 a 9 e no *Material de apoio*.

O que você define e o que já vem pronto

Você define

1. A **API key** do grupo
   ponte, variável `apikey` e firmware
2. Corpo do **grupo de serviço**
   requisição 2.1
3. Corpo do **dispositivo**
   requisição 2.2
4. Corpo da **subscrição do Cygnus**
   requisição 4.1
5. Corpo da **notificação para o serviço externo**
   requisição 4.4

Além disso: os **TODO do firmware** (Etapa 3) e o **painel** no Grafana (Etapa 4).

Já vem pronto

Exemplo resolvido: uma lâmpada inteligente simulada, com todas as requisições prontas.

Ambiente: `plataforma/docker-compose.yml`, MySQL, consumidor e a **ponte** (você só troca a API key).

Todas as outras requisições da coleção: consultas, os **comandos** da Etapa 3 e a simulação de solo seco.

Verificações de todas as etapas: as requisições **[Verificação]**.

Reset do provisionamento: pasta *0 - Recomeçar do zero*.

Reset completo: `docker compose down -v`.

A pasta do projeto é o seu repositório

```
projeto2-<Nome>-<apikey>/
├── README.md                    ← documentação do grupo
├── relatorio.pdf                ← VOCÊ ACRESCENTA
├── documentacao apoio/          (guia, material de apoio)
├── firmware/
│   └── main_esqueleto.py        ← TODO 1 a 5
├── plataforma/
│   ├── docker-compose.yml       (pronto)
│   ├── mosquitto/mosquitto.conf  ← sua API key
│   ├── mysql/                   (pronto)
│   └── servicos/                (prontos, com a lâmpada)
└── postman/
    ├── Exemplo - Lâmpada...json   (pronta)
    └── Projeto 2...json         ← apikey + 4 corpos
```

Regras

Não altere a estrutura de pastas nem os nomes dos arquivos.

Altere **só o conteúdo** dos arquivos marcados com ←.

Coloque o **relatório em PDF na raiz** da pasta.

Exporte a coleção por cima de `postman/Projeto 2.postman_collection.json` (Collection v2.1) e salve o painel do Grafana como `plataforma/painel-estufa.json`.

Identificadores do projeto

Item	Valor	Observação
API key	definida pelo grupo	letras e números, **única na turma**; maiúsculas contam
Client id da ponte	`ponte_<API key>`	no broker público, todo client id precisa ser único
`fiware-service` / `servicepath`	`fei` / `/estufa`	os mesmos no IoT Agent e no Orion
`device_id`	`estufa001`	o mesmo no provisionamento, na ponte e no firmware
Entidade	`urn:ngsi-ld:Greenhouse:001`	tipo `Greenhouse`
Client id da ESP32	`"esp32_" + API_KEY + "_" + DEVICE_ID`	já montado no esqueleto do firmware

Anote esses valores no `README.md` do repositório antes de começar.

1. Etapa 1 – Plataforma FIWARE

API key, ambiente no ar e a ponte com o broker público.

1.1 – Pré-requisitos

1. **Docker Desktop aberto**
   Encerre os ambientes das Aulas 8 e 9 (`docker compose down` em cada pasta).

2. **Imagens baixadas antes da aula**
   `docker compose pull` na pasta `plataforma` (cerca de 4 GB).

3. **Postman e um cliente MQTT**
   HiveMQ WebSocket Client (o do Projeto 1) ou MQTTX.

4. **A estufa do Projeto 1**
   ESP32 com Thonny **ou** o projeto no Wokwi.

Sempre na pasta plataforma

Todos os comandos `docker compose` deste guia rodam **dentro** da pasta `plataforma`, onde está o `docker-compose.yml`.

```
cd plataforma
```

Portas livres: 1026, 27017, 4041, 7896, 1883, 3000, 3001, 3306, 5050 e 5080.

1.2 – Defina a API key do grupo

Onde	O que trocar
`plataforma/mosquitto/mosquitto.conf`	`profRafael` → a sua API key no `remote_clientid` e nas 3 linhas `topic`
`postman/Projeto 2.postman_collection.json`	variável da coleção `apikey` (aba *Variables*)
`firmware/main_esqueleto.py`	constante `API_KEY`

Regras da API key

Única na turma e **igual** nos três lugares. Letras e números; maiúsculas e minúsculas contam.

Não use profRafael

É a API key de exemplo do professor. Duas estufas com a mesma API key misturam medidas e comandos.

1.3 – A ponte com o broker público

```
connection hivemq
address broker.hivemq.com:1883
remote_clientid ponte_<API key>
cleansession true
try_private false                  # não altere
notifications false                # não altere
bridge_protocol_version mqttv311   # não altere

topic /json/<API key>/estufa001/attrs  in  0
topic /json/<API key>/estufa001/cmdexe in  0
topic /<API key>/estufa001/cmd         out 0
```

`plataforma/mosquitto/mosquitto.conf` já vem pronto: troque **só** a API key. O `docker-compose.yml` não precisa de alteração.

Tópico	Quem publica	Direção
`.../attrs`	estufa	`in`
`.../cmdexe`	estufa	`in`
`.../cmd`	IoT Agent	`out`

Entenda as direções

`in`: do broker **público** para o **local**, onde está o IoT Agent.

`out`: do broker **local** para o **público**, onde está a estufa.

Só os tópicos da sua API key atravessam a ponte.

1.4 – Suba o ambiente

```
cd plataforma
docker compose up -d
docker compose ps        # 9 contêineres healthy
docker compose logs mosquitto
```

No log do Mosquitto:

`Connecting bridge hivemq` **uma vez**: a ponte conectou.

A mesma linha **repetindo** + `closed its connection`: client id repetido ou linhas fixas alteradas.

`Error resolving bridge address`: endereço do broker errado.

Por que não direto?

O IoT Agent assina **curingas** em um grupo `$share/json` de nome fixo. Ligado ao broker público, ele dividiria as medidas com os IoT Agents da turma, **mesmo com API keys diferentes**.

1.5 – Teste a ponte nos dois sentidos

```
docker exec -it mosquitto mosquitto_sub -v -t "#"     # osciloscópio do broker LOCAL
```

#	Faça	Resultado esperado
1	No **HiveMQ Web Client**, publique `oi` no tópico de **medidas** da sua estufa	aparece no terminal local
2	No Web Client, publique no mesmo tópico, trocando a API key por outra qualquer	**não** aparece no terminal local
3	No broker **local**, publique no tópico de **comandos** da sua estufa (com o Web Client assinando os comandos)	aparece no Web Client

```
docker exec mosquitto mosquitto_pub -t "<tópico>" -m oi
```

O teste 1 ou o 3 falhou? Confira se a API key é a mesma no `mosquitto.conf` e no tópico.

1.6 – Postman e ponto de verificação 1

1 **Importe a coleção**
`postman/Projeto 2.postman_collection.json`

2 **Preencha a variável** `apikey`
Coleção → aba *Variables* → a sua API key → *Save*. Use `{{apikey}}` nos corpos.

3 **Rode a pasta Etapa 1 - Ambiente**
Botão *Run* na pasta. As requisições **[Verificação]** já estão prontas.

✓ Ponto de verificação

- ☐ `docker compose ps`: 9 contêineres healthy
- ☐ A ponte conectou uma única vez
- ☐ Os 3 testes da ponte deram certo
- ☐ Todos os testes da pasta **Etapa 1** passam

1.7 – Como recomeçar do zero

1 Só o provisionamento

Pasta **0 – Recomeçar do zero** da coleção (pronta).

Apaga subscrições, dispositivo, entidade e grupo do seu caminho.

O **histórico** (MySQL) e o **painel** (Grafana) continuam.

2 Tudo: docker compose down -v

Remove os **serviços** e os **volumes** deste compose: entidades, subscrições, grupos e dispositivos (MongoDB), histórico (MySQL), fonte de dados e painéis (Grafana).

Depois, `docker compose up -d` sobe um ambiente limpo.

```
cd plataforma
docker compose down -v   # serviços + volumes
docker compose up -d     # ambiente limpo
```

Antes do `down -v`, **exporte o painel** do Grafana. Sem o `-v`, o `docker compose down` para os serviços e **mantém** os dados.

2 Exemplo resolvido – Lâmpada inteligente

Veja o sistema funcionando antes de montar a estufa.

E.1 – A lâmpada inteligente simulada

```
cd plataforma
docker compose ps      # lampada-simulada healthy
docker compose logs -f lampada-simulada
```

Item	Lâmpada
API key · caminho	`lampada2026` · `/lampada`
Dispositivo · entidade	`lampada001` · `urn:ngsi-ld:Lamp:001`
Medidas (`object_id`)	`s` → `state` (0/1) · `l` → `luminosity` (%)
Comando · estático	`setLight` (0/1) · `name`

Tudo pronto

Simulador em `plataforma/servicos/lampada_simulada.py` e coleção `postman/Exemplo - Lâmpada.postman_collection.json`.

Sobe junto com o ambiente e fala com o Mosquitto **local**: sem ponte e sem colidir com os colegas.

Outra API key e outro caminho: **não interfere** na estufa.

O mesmo IoT Agent, Orion, Cygnus e consumidor que a estufa vai usar.

E.2 – Rode e leia o exemplo

1. **Rode a coleção inteira no Runner**
   `postman/Exemplo - Lâmpada.postman_collection.json` (cerca de 1 min). Tudo verde.

2. **Leia os 4 corpos**
   1.1 (grupo de serviço), 1.2 (dispositivo), 3.1 (Cygnus) e 3.3 (notificação externa). São os que você vai escrever para a estufa.

3. **Acompanhe um comando**
   Rode 2.1 de novo e veja o comando e a resposta no log do simulador e no `mosquitto_sub`.

4. **Confira o histórico**
   No MySQL, a tabela `lampada` do banco `fei`.

Pasta	O que mostra
0	reset do exemplo
1	grupo, dispositivo e registro
2	comando válido e inválido
3	Cygnus e notificação

Cada requisição tem uma **descrição** que explica o corpo e termina com **Na estufa**: o que muda no seu caso.

E.3 – Da lâmpada para a estufa

Você define	Lâmpada (pronto)	Estufa (você faz)
API key	`lampada2026`, só no broker local	única na turma, na ponte, na coleção e no firmware
Grupo de serviço	tipo `Lamp`	tipo `Greenhouse`
Dispositivo	2 medidas, 1 comando, `name`	7 medidas, 4 comandos, `name` + `location` (geo:json)
Subscrição do Cygnus	`Lamp`, 2 medidas → tabela `lampada`	`Greenhouse`, 7 medidas → tabela `estufa`
Notificação externa	`state==1;luminosity>70` → `/alerta-lampada`	`soilMoisture` < 30 → `/alerta-estufa`

E o firmware? Compare `lampada_simulada.py` (Python) com o esqueleto da estufa (MicroPython): tópicos, JSON de medidas, callback e resposta no `cmdexe`.

3. Etapa 2 – Modelo de dados e provisionamento

Os corpos do grupo de serviço e do dispositivo.

2.1 – O modelo da estufa

Medida	Atributo	Faixa
Temperatura do ar	`temperature`	°C
Umidade do ar	`relativeHumidity`	%
Umidade do solo	`soilMoisture`	% calibrada
Luminosidade	`illuminance`	% calibrada
Relé	`relayState`	0 ou 1
Janela (servo)	`windowAngle`	0 a 180
Intervalo	`samplingInterval`	2 a 10 s

Comando	Valor
`setRelay`	0 ou 1
`setServo`	0 a 180
`setInterval`	2 a 10
`setSending`	1 / 0

Estáticos: `name` (Text) e `location` (geo:json, Point).

Os **nomes dos atributos** são fixos. Os **object_id** (chaves curtas do JSON) são escolha do grupo: escreva a tabela `object_id → atributo` no README **antes** de provisionar.

2.2 – Corpo do grupo de serviço

Requisição 2.1 · Complete o corpo

```
POST http://{{iotagent}}/iot/groups
fiware-service: {{servico}}
fiware-servicepath: {{caminho}}
Content-Type: application/json

# Body (JSON): VOCÊ ESCREVE
```

Método, URL e cabeçalhos já estão prontos: **não os altere**.

Os testes conferem o grupo **no IoT Agent**, não só a resposta.

O corpo precisa definir

a **API key**: use `{{apikey}}`;

o tipo de entidade `Greenhouse`;

o recurso `/iot/json`;

o Orion **visto de dentro** da rede Docker;

que dispositivos **não provisionados** sejam **rejeitados** (R3).

Aula 8, README seção 8 (API de provisionamento).

2.3 – Corpo do dispositivo

Requisição 2.2 · Complete o corpo

```
POST http://{{iotagent}}/iot/devices
fiware-service: {{servico}}
fiware-servicepath: {{caminho}}
Content-Type: application/json

# Body (JSON): VOCÊ ESCREVE
```

Use as variáveis da coleção: `{{device_id}}`, `{{apikey}}` e `{{entidade}}`.

Os **object_id** precisam ser **os mesmos** do firmware.

O corpo precisa definir

`device_id`, **API key**, entidade e tipo `Greenhouse`;

o transporte **MQTT**;

os **7 atributos** de medida (`object_id` → nome, Number);

os **4 comandos**;

os **2 estáticos**: `name` e `location`.

Modelo da estufa: slide 2.1 · Aula 8, README seção 8.

2.4 – Registro e medida de teste

1. **Rode 2.3 [Pronto]**
   Consulta os registros no **Orion**: quem atende os comandos? E por que a entidade só tem os comandos? (R5)

2. **Assine os tópicos da sua API key**
   No HiveMQ Web Client, com o curinga #. É o seu "osciloscópio" no broker público.

3. **Publique uma medida no tópico de medidas**
   Um JSON com uma ou duas chaves suas e valores **numéricos**.

4. **Rode 2.4 [Verificação]**
   Chegou ao `mosquitto_sub` local, mas não ao Orion? Veja `docker compose logs iot-agent`.

Sentido	Tópico (IoT Agent JSON)
Medidas	`/json/<apikey>/<device_id>/attrs`
Comandos	`/<apikey>/<device_id>/cmd`
Resultado	`/json/<apikey>/<device_id>/cmdexe`

✓ Ponto de verificação

- ☐ Testes **2.1** a **2.4** passam
- ☐ Tabela `object_id → atributo` no README

4 Etapa 3 – Firmware da estufa

TODO 1 a 5 do esqueleto, testados pelas requisições prontas.

3.1 – Prepare o firmware

1. **Complete o esqueleto**
   `firmware/main_esqueleto.py` (no repositório o nome não muda). Na ESP32 ou no Wokwi, grave como `main.py`.

2. **Preencha a configuração**
   `SSID`, `SENHA` e `API_KEY` (a mesma da ponte). No Wokwi: `Wokwi-GUEST`, sem senha.

3. **Traga do Projeto 1**
   Leitura e calibração dos sensores (`TODO 2a`) e a conversão do servomotor.

Wokwi

A ESP32 simulada acessa a internet e alcança o broker público: dá para fazer o projeto inteiro sem a placa.

Nunca no GitHub

A senha do Wi-Fi **não** pode ir para o repositório. Troque por um valor de exemplo antes do *commit*.

3.2 – Medidas: TODO 1 e TODO 2

1. **TODO 1 · tópicos**
   Os três tópicos a partir de `API_KEY` e `DEVICE_ID`.

2. **TODO 2 · medidas**
   `ler_sensores()`, `publicar_medidas()` (UM JSON, 7 medidas) e o laço principal.

3. **Rode e confira, nesta ordem**
   Shell → HiveMQ Web Client → `mosquitto_sub` local → Postman **3.1 [Pronto]**.

Onde a mensagem parou?

Shell sem JSON → problema no firmware (sensores, laço).

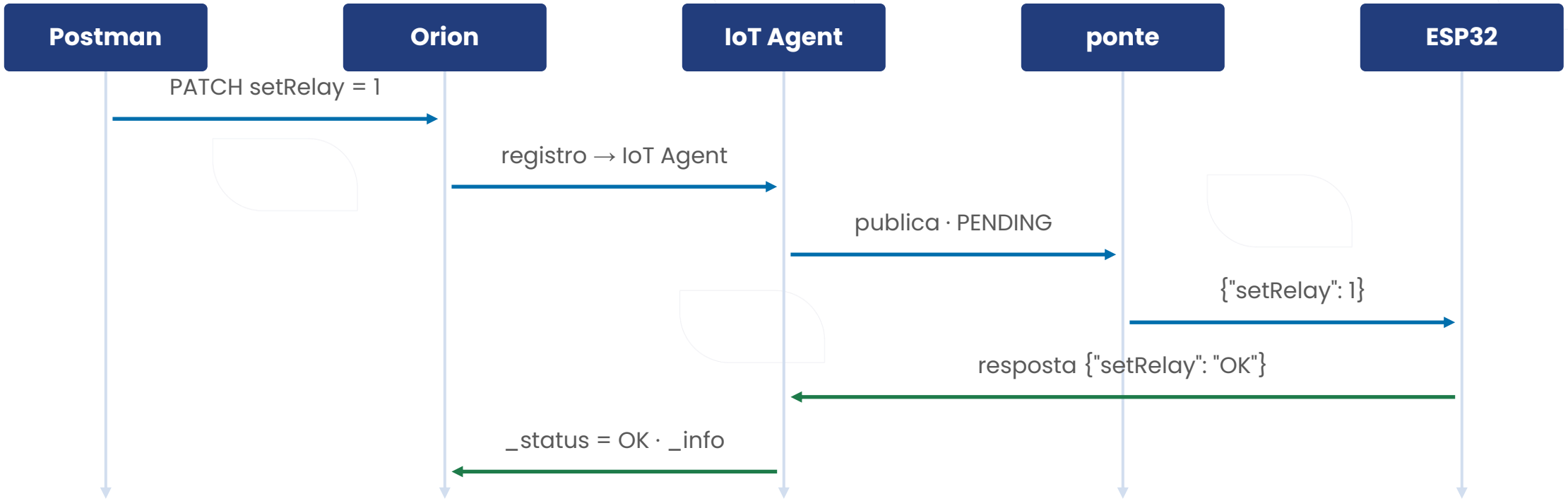
Web Client sem mensagem → tópico, Wi-Fi ou conexão MQTT.

mosquitto_sub local sem mensagem → API key do tópico diferente da ponte.

Orion sem atualização → `device_id`, provisionamento ou JSON inválido (log do IoT Agent).

Atributos `t`, `s`... no Orion → chaves diferentes dos object_id.

3.3 – Comandos: TODO 3 e TODO 4



TODO 3: `setRelay` e `setServo` com validação. **TODO 4:** callback, resposta no tópico de resultado e assinatura do tópico de comandos. As requisições **3.2 a 3.7 já estão prontas**: rode-as e acompanhe o comando nos dois lados da ponte.

3.4 – Modos de operação: TODO 5

1. `setInterval`
   2 a 10 s. Fora da faixa: `ERRO`, e o intervalo **não** muda.

2. `setSending`
   0 para as medidas; os comandos continuam funcionando (`R6`).

3. **Requisições 3.8 a 3.15 (prontas)**
   A 3.13 leva ~16 s: ela espera para provar que **nenhuma** medida chegou.

4. **Teste de robustez**
   Publique `teste` no tópico de comandos pelo Web Client: nada pode cair.

✓ Ponto de verificação

- ☐ Todos os testes da pasta **Etapa 3** passam
- ☐ Checklist do fim de `main_esqueleto.py` completo
- ☐ Sem `time.sleep()` e sem `wait_msg()` no laço
- ☐ Senha do Wi-Fi fora do repositório

5. Etapa 4 – Consumo do contexto

Os corpos das duas subscrições e o painel.

4.1 – Corpo da subscrição do Cygnus

Requisição 4.1 · Complete o corpo

```
POST http://{{orion}}/v2/subscriptions
fiware-service: {{servico}}
fiware-servicepath: {{caminho}}
Content-Type: application/json

# Body (JSON): VOCÊ ESCREVE
```

Depois: **4.2** e **4.3 [Verificação]** e confira o MySQL:

```
docker exec -it db-mysql mysql -u root -pfei2026 fei -e \
  "SELECT recvTime, attrName, attrValue FROM estufa
   ORDER BY CAST(recvTimeTs AS UNSIGNED) DESC LIMIT 14;"
```

O corpo precisa definir

as entidades do tipo `Greenhouse`;

só os 7 atributos de medida;

o endereço de notificação do **Cygnus**, visto por **quem envia** (o Orion, na rede Docker);

o formato que o Cygnus aceita.

Aula 9, README seção 5.

Errou? Apague a subscrição errada (pasta 0 ou `DELETE`): senão ela continua gravando.

4.2 – Corpo da notificação para o serviço externo

Requisição 4.4 · Complete o corpo

```
POST http://{{orion}}/v2/subscriptions
fiware-service: {{servico}}
fiware-servicepath: {{caminho}}
Content-Type: application/json

# Body (JSON): VOCÊ ESCREVE
```

Depois: **4.5** a **4.8** (prontas). A **4.7** simula uma medida de solo seco pela porta HTTP do IoT Agent.

Ao vivo: tire o sensor da terra (ou gire o potenciômetro no Wokwi) e veja `localhost:3001/notificacoes?caminho=/alerta-estufa`.

O corpo precisa definir

a entidade da estufa;

a condição: `soilMoisture` **abaixo de 30 %** (expressão `q`);

o serviço externo: `http://consumidor:3000/alerta-estufa`;

`keyValues`, só com `soilMoisture`, `relayState` e `TimeInstant`;

sem uma notificação a cada medida enquanto o solo estiver seco.

Aula 7 · Material de apoio, seção 6.

4.3 – Painel no Grafana

1. **Fonte de dados MySQL**
   Usuário **somente leitura** · verificações 4.9 e 4.10 (Aula 9).

2. **Histórico no MySQL**
   Verificação 4.11: só os 7 atributos nos últimos 30 s.

3. **Painel "Estufa inteligente"**
   a) temperatura e umidade · b) solo · c) relé (*State timeline*) · d) luminosidade (*Stat*/*Gauge*) · atualização automática.

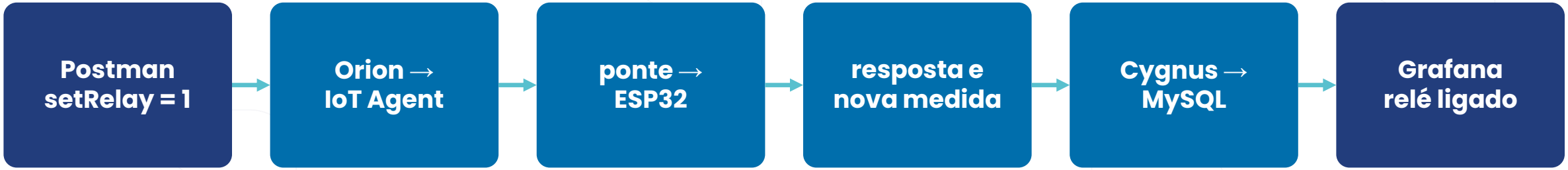
4. **Verificações 4.12 e 4.13**
   Título exato e conteúdo do painel.

```
SELECT CAST(recvTime AS DATETIME(3)) AS time,
  CAST(attrValue AS DECIMAL(6,1)) AS '<legenda>'
FROM estufa
WHERE attrName = '<atributo>'
  AND $__timeFilter(recvTime)
ORDER BY time;
```

Padrão para **uma** série: monte os outros painéis a partir dele.

Modo **Code**, formato *Time series*, e nunca `CAST` dentro de `$__timeFilter`.

4.4 – Fechando o ciclo



Esse é o **roteiro da demonstração final**: um comando pelo Orion, a resposta no Web Client, `_status/_info` no Orion e o painel reagindo.

Antes de entregar: rode a **coleção inteira** no Runner com a estufa ligada. A pasta **0** apaga o provisionamento e as suas requisições recriam. Exporte o resultado para `postman/`.

✓ Ponto de verificação

- ☐ Todos os testes da pasta **Etapa 4** passam
- ☐ **Coleção inteira** no Runner, sem falhas
- ☐ Painel exportado em `plataforma/painel-estufa.json`

6. Desafios, entrega e problemas comuns

Bônus, repositório, demonstração e diagnóstico.

Desafios (opcionais, bônus)

D.1 · Automação em malha fechada

Um serviço recebe o alerta e liga a irrigação **pelo Orion**; outra subscrição desliga acima de 60 %. Nada de MQTT no serviço.

D.2 · Reconexão

O firmware detecta a queda da conexão (`OSError`) e reconecta sozinho, assinando de novo os comandos.

D.3 · TLS

Estufa e ponte na porta TLS do broker público. O que o TLS resolve e o que **não** resolve sem autenticação?

D.4 · Segunda estufa

`estufa002` (outra instância no Wokwi), com os tópicos dela na ponte, e as duas no painel, uma série por estufa.

Entrega e demonstração

O que entregar

A **pasta do projeto** como repositório `projeto2-<Nome>-<apikey>`, com a **mesma estrutura** do material.

`README.md` preenchido (documentação do grupo).

`relatorio.pdf` **na raiz**, com as respostas **R1 a R8**.

Coleção exportada em `postman/` e o resultado do Runner.

Painel em `plataforma/painel-estufa.json`.

Firmware completo, **sem** a senha do Wi-Fi.

Roteiro da demonstração (5 min)

1. Ambiente no ar e a estufa publicando (Web Client + Orion).
2. Um comando de cada tipo pelo Orion, incluindo um inválido.
3. `setSending` = 0 e = 1.
4. Alerta de solo seco no consumidor.
5. Painel reagindo a um comando.
6. Perguntas sobre o caminho dos dados (R4 e R5).

Perguntas para o relatório

#	Pergunta (texto completo no guia em Markdown, seção 7)
R1	Assinatura compartilhada: o que aconteceria sem a ponte? Por que cada tópico da ponte tem a sua direção?
R2	Por que a API key precisa ser única na turma?
R3	Por que rejeitar dispositivos não provisionados no broker público?
R4	O caminho de uma medida, da ESP32 ao Grafana: componente, protocolo, porta e formato.
R5	O caminho de um `setRelay` e de volta. Por que `_status` = OK não garante que a estufa aceitou?
R6	Por que `setSending` = 0 não pode desligar a escuta de comandos?
R7	Orion × MySQL. Por que `normalized` para o Cygnus e `keyValues` para o alerta?
R8	Dois ataques possíveis à sua estufa no broker público e duas medidas que os evitariam.

Problemas comuns

Sintoma	Causa e solução
`mosquitto` unhealthy, IoT Agent não sobe	Erro no `mosquitto.conf`: `docker compose logs mosquitto`.
`Connecting bridge` repetindo	Client id da ponte repetido ou linhas fixas alteradas; ou porta 1883 bloqueada na rede.
No Web Client, mas não no `mosquitto_sub`	API key do tópico diferente da usada na ponte.
No `mosquitto_sub`, mas não no Orion	`device_id`, corpo do provisionamento ou JSON inválido: log do IoT Agent.
Teste de uma requisição pronta falhou	Variável `apikey` diferente da ponte e do firmware, ou corpo 2.1/2.2 incompleto.
Comando fica `PENDING`	ESP32 sem assinar ou sem responder, ou chave da resposta diferente do comando.
Nada mais faz sentido	Recomece do zero: `docker compose down -v` e `docker compose up -d`.

Obrigado

CCM520 – Projeto 2 · Guia passo a passo

Coordenador da disciplina
Rafael Gomes Alves
ralves@fei.edu.br

JESUÍTAS BRASIL